

Your Essential Guide to Success

 systemdesignschool.io/fundamentals/what-is-system-design-interview



The System Design Interview Course

What Is a System Design Interview?

System design is the process of architecting and planning (often data-intensive) applications by carefully selecting and integrating various data to efficiently handle data processing and storage needs. This involves understanding the requirements of the application, identifying the appropriate data systems to manage the diverse tasks, and using application code to stitch together these different tools, allowing them to work in harmony.

When you integrate multiple tools to deliver a service, the service's interface or API typically conceals the implementation specifics from clients. By doing this, you have effectively constructed a new, specialized data system from smaller, general-purpose components. This combined data system can offer certain assurances, such as maintaining cache consistency through proper invalidation or updates, ensuring that external clients receive consistent results.

System Design vs OOP Design

	OOP Design	System Design
Example	Design a parking lot, design a vending machine	Design WhatsApp, Design Uber
Skills tested	Writing well-structured OOP code, class inheritance	Architecture, dataflow, scalability, design tradeoffs

	OOP Design	System Design
Interviewees asked to	Writing OOP code, class interface, and inheritance, etc.	Architecture diagrams, storage design, resource estimates, design discussion, tradeoffs, deep dive, etc.

Who Needs to Go Through System Design Interviews?

System design interviews are essential for various roles in the software engineering field. They become increasingly important as engineers progress in their careers and take on more complex projects or leadership positions. The following professionals are likely to encounter system design interviews:

- **Experienced Software Development Engineer (SDE) or Software Engineer (SWE):** As engineers gain experience, they are often tasked with designing and architecting complex systems, making system design skills crucial. These interviews assess their ability to break down problems, select appropriate technologies, and create scalable and efficient solutions. At this level, a tech interview is often a combination of LeetCode-style questions and system design questions.
- **Engineering Manager (EM, SDM):** Engineering managers are responsible for leading and guiding teams of engineers in the development and execution of software projects. They need to have a strong understanding of system design principles to make informed decisions and ensure the successful delivery of projects. System design interviews help evaluate their technical knowledge and leadership skills.
- **Technical Program Manager (TPM):** TPMs are responsible for overseeing the planning, execution, and delivery of technical projects. They need to understand the intricacies of system design to coordinate resources and manage project risks effectively. System design interviews can help assess their technical expertise and ability to manage complex projects.

What Are We Designing and What Skills Do I Need?

System design is the process of designing a complex system to meet specific requirements and goals, typically aimed at providing a service to the end user. In order to achieve this, we assemble various technologies and components to create a cohesive and functional system.

When we engage in system design, we focus on designing the following aspects:

- **Services:** This includes the various components, applications, and APIs that provide specific functionality or processing capabilities. Services can be designed as monoliths or, more commonly, as microservices to enable better scalability, maintainability, and flexibility.
- **Dataflow:** Designing the flow of data within and between the services is crucial for ensuring efficient processing, timely communication, and accurate results. This involves understanding the data formats, protocols, and communication patterns used between services.

- **Storage:** The storage design encompasses databases, caches, and file systems required to store and manage data throughout the system. This includes selecting appropriate storage technologies, data modeling, and ensuring data consistency and durability.

Functional Requirements

Functional requirements are the features and capabilities that a system must have to fulfill its intended purpose. They describe what a system is supposed to do and are focused on the specific tasks or functionalities the system should perform. Functional requirements are often derived from user needs, business objectives, or system specifications.

Examples of functional requirements include:

1. **User actions:** Describing the actions a user can perform within the system, such as creating an account, logging in, or submitting a form.
2. **Data input and processing:** Defining how the system should process, manipulate, or transform data based on user input or other sources.

Functional requirements are usually documented and communicated to developers and stakeholders through various formats, such as [project requirement docs](#) (PRDs). These requirements help guide the development process and ensure that the final product meets the intended objectives and provides the desired functionality.

Non-functional Requirements

Besides making things functionally work, we also have to make sure the system satisfies non-functional requirements, most notably:

- **Scalability:** A well-designed system should be able to handle increasing amounts of work or users without compromising performance. This involves designing for horizontal and vertical scaling, optimizing resource usage, and planning for future growth.
- **Availability:** High availability is essential for ensuring that the system can continue to function even in the face of failures, such as hardware or network issues. This requires designing for redundancy, failover mechanisms, and monitoring the system's health.
- **Performance (latency & throughput):** Latency refers to the time taken to respond to requests, while throughput is the amount of work or transactions the system can handle in a given time frame.

Besides these main non-functional requirements, there are also reliability (the service returns correct results), consistency (data is consistent between services), and efficiency (the service should have minimal redundant operations).

What Skills Do I Need to Excel My System Design Interviews?

To excel in system design, engineers need to develop a broad set of skills, including:

- Deep technical knowledge of various technologies and components, such as databases, messaging systems, and batch processing systems.
- Architectural and design pattern knowledge to identify reusable and scalable approaches.
- An understanding of trade-offs and the ability to make informed decisions based on constraints and requirements.
- Strong analytical and problem-solving skills to break down complex requirements and design effective solutions.
- And finally, a candidate needs effective communication skills to collaborate with team members and stakeholders and articulate design decisions.

What Makes System Design Interviews Hard to Prepare?

Preparing for system design interviews can be difficult due to several reasons:

- Lack of formal education: Most university courses focus on theoretical concepts rather than practical experience, making it hard for students to learn system design from their academic curriculum. There isn't really an academic course you can take on system design.
- Limited exposure to large-scale systems: In many workplaces, engineers mainly work on connecting APIs or managing smaller-scale projects. This lack of experience with large-scale systems makes it harder to grasp the intricacies of system design.
- Questionable quality in online resources: The reliability of information in free content like blogs can be questionable, making it difficult to gauge the correctness of the material.
- Abstract company-specific engineering blogs: A company's engineering blog can provide insights into their systems. However, they often assume prior knowledge and are very hard to read.
- Disorganized learning resources: Many courses and books on system design lack a structured approach, which can lead to shallow or disorganized content. This often leaves learners unsure of how to apply the knowledge to design a system from scratch.
- Memorization and buzzword stacking: Due to the absence of a clear learning path, many candidates resort to memorizing buzzwords or specific solutions, which may not be helpful in solving new or unique problems.

To overcome these challenges, it's crucial to seek out comprehensive and structured learning resources, gain hands-on experience through personal projects or internships, and practice problem-solving using first-principles thinking. This will help you develop a deeper understanding of system design concepts and enable you to tackle new problems more effectively during interviews.

What Do I Get from Completing This Course?

Our promise to you is that if you go through all the material in this course, you will get a thorough understanding of the key components and concepts of system design, being able to reason from first principles to make tradeoff decisions to design systems from scratch.

This is the most comprehensive course out there on system design [period].